

Transitions: DS + Algorithms

Divide &
Conquer
Recurrences



First: worksheet!

$$V(n) = \# \text{ computed} = 4V(n-1) - 4V(n-2)$$

$$T(n) = \text{runtime} = T(n-1) + T(n-2) + O(1) \\ (\text{or } +4)$$

Quicksort;

Solving:

```

QUICKSORT(A[1..n]):
  if (n > 1)
    Choose a pivot element A[p]
    r ← PARTITION(A, p)
    QUICKSORT(A[1..r-1])  <<Recurse!>>
    QUICKSORT(A[r+1..n]) <<Recurse!>>
  
```

```

PARTITION(A[1..n], p):
  swap A[p] ↔ A[n]
  ℓ ← 0  <<#items < pivot>>
  for i ← 1 to n-1
    if A[i] < A[n]
      ℓ ← ℓ + 1
      swap A[ℓ] ↔ A[i]
  swap A[n] ↔ A[ℓ + 1]
  return ℓ + 1
  
```

Figure 1.8 Quicksort

worst case!

this depends on pivot

$$T(n) \leq \max_{1 \leq r \leq n} \{ T(r-1) + T(n-r) + O(n) \}$$

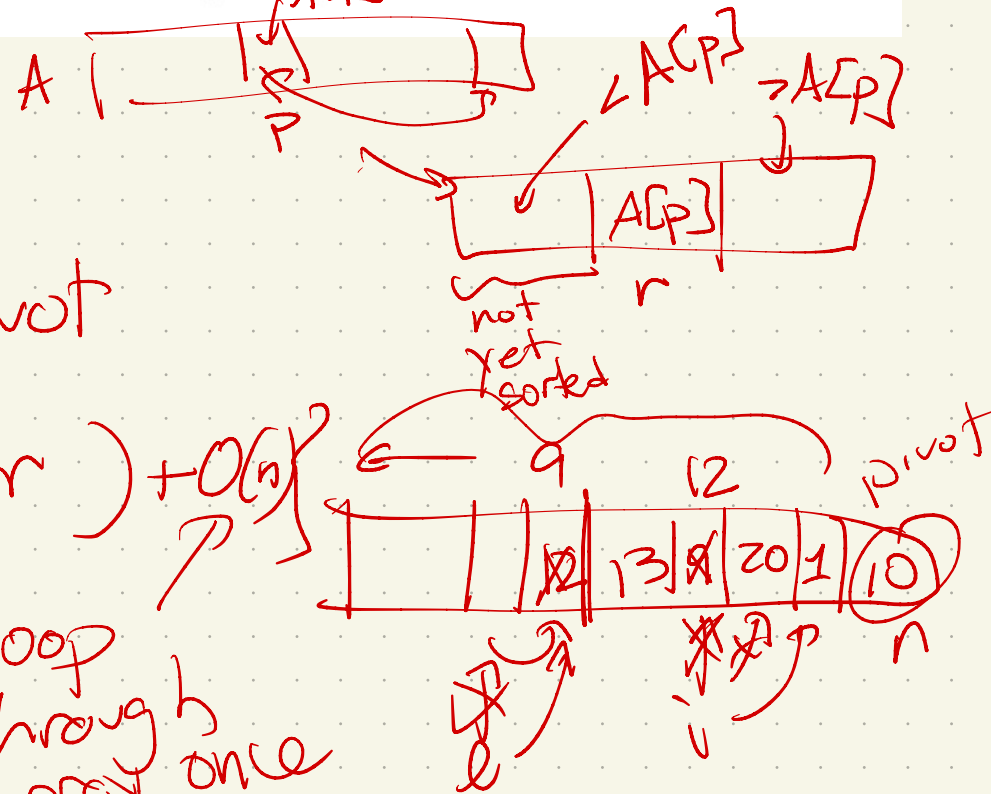
loop through array once

worst case:

pivot is smallest or largest

$$T(n) = T(n-1) + O(n)$$

$$T(n) = n + (n-1) + (n-2) + \dots = n^2$$



If lucky:
choose middle element

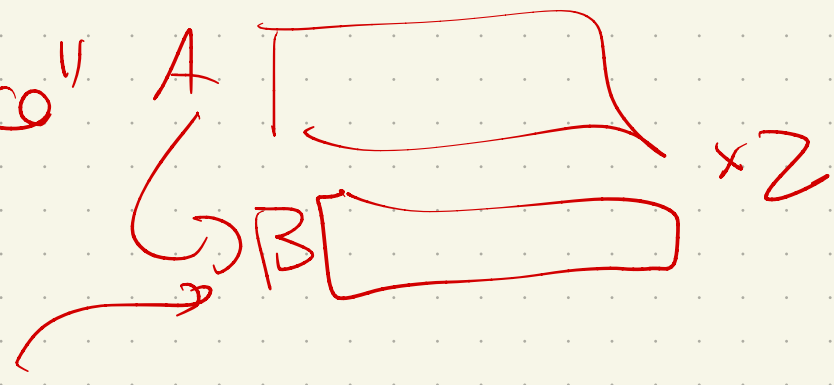
$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(n) \\ = 2T\left(\frac{n}{2}\right) + n$$

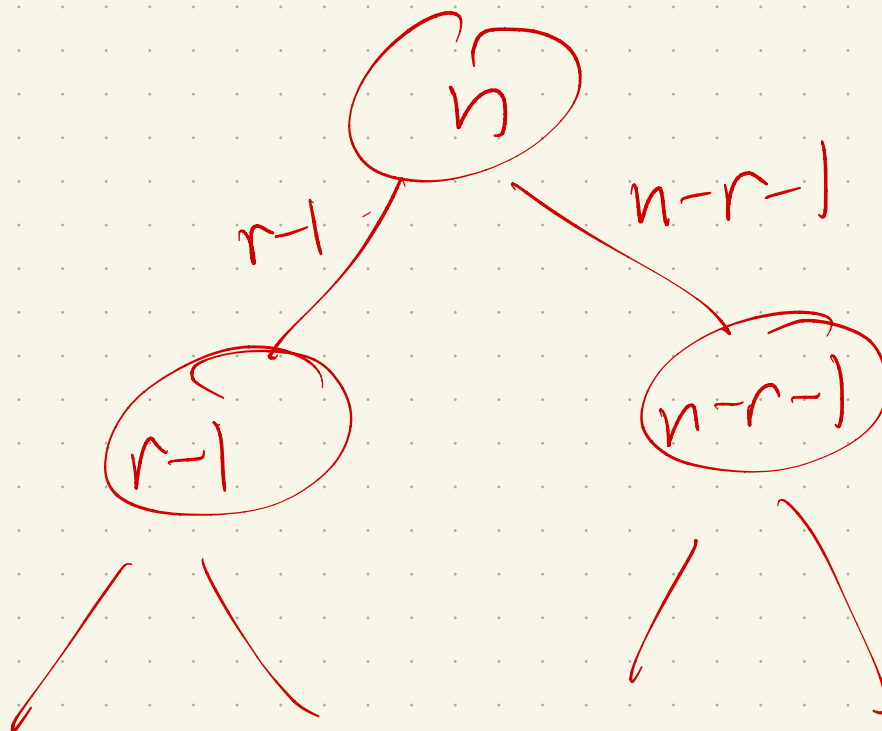
→ $n \log n$

Other factor: space

Quicksort is "in place"

Mergesort uses
 $O(n)$ extra space





Runtime:

MERGESORT(A[1..n]):

if $n > 1$

$m \leftarrow \lfloor n/2 \rfloor$

$\rightarrow$ MERGESORT(A[1..m]) $\llcorner$ Recurse! $\llcorner$

$\rightarrow$ MERGESORT(A[m+1..n]) $\llcorner$ Recurse! $\llcorner$

MERGE(A[1..n], m)

$\hookrightarrow O(n)$

MERGE(A[1..n], m):

$i \leftarrow 1; j \leftarrow m+1$

for $k \leftarrow 1$ to n

if $j > n$

$B[k] \leftarrow A[i]; i \leftarrow i+1$

else if $i > m$

$B[k] \leftarrow A[j]; j \leftarrow j+1$

else if $A[i] < A[j]$

$B[k] \leftarrow A[i]; i \leftarrow i+1$

else

$B[k] \leftarrow A[j]; j \leftarrow j+1$

for $k \leftarrow 1$ to n

$A[k] \leftarrow B[k]$

Figure 1.6. Mergesort

$\sum_{k=1}^n 5 = 5n$
 $= O(n)$
 $\sum_{k=1}^n 1 = n$
 $O(n) = \sum_{k=1}^n 1$

Let $T(n)$ = runtime of MS on
input of size n

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lfloor \frac{n}{2} \rfloor) + O(n)$$

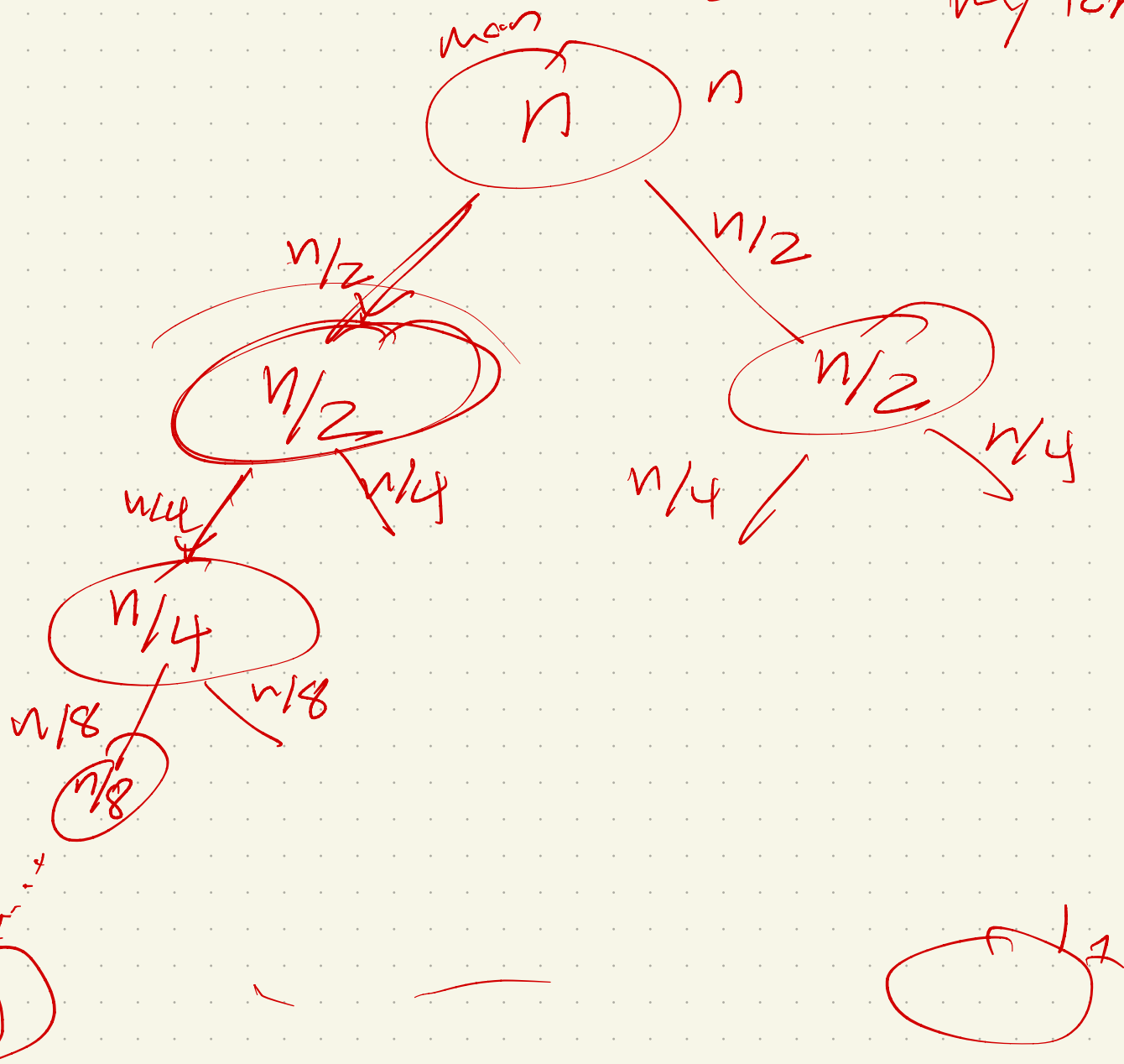
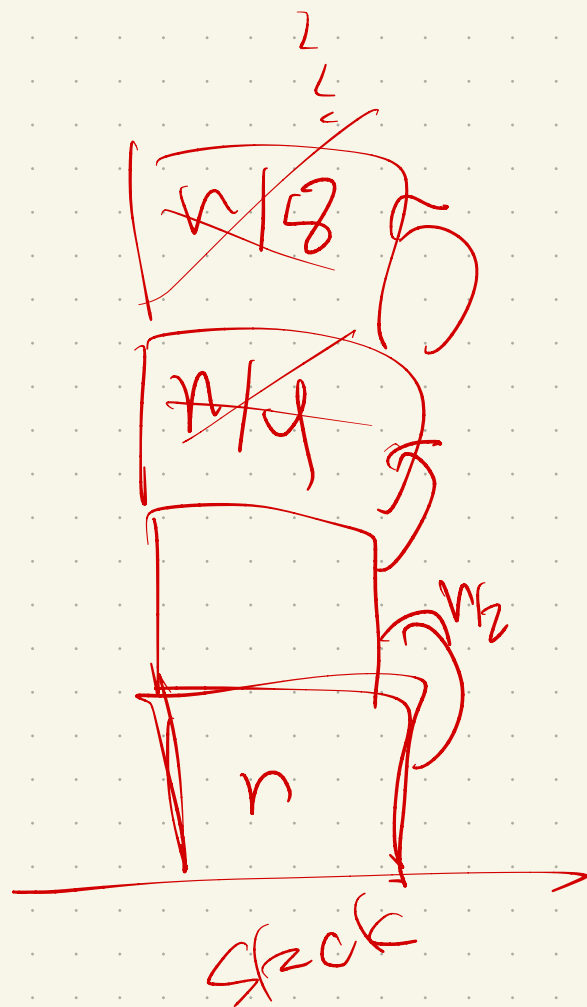
$$\hookrightarrow T(n) = 2T(\frac{n}{2}) + n$$

Recursion "tree":

$$T(n) = 2T(n/2) + n$$

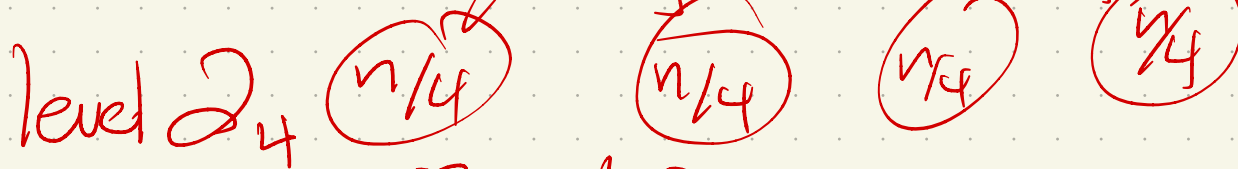
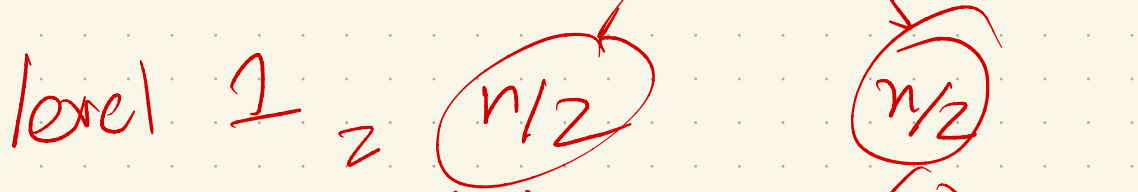
2 rec calls

work
in
my fun

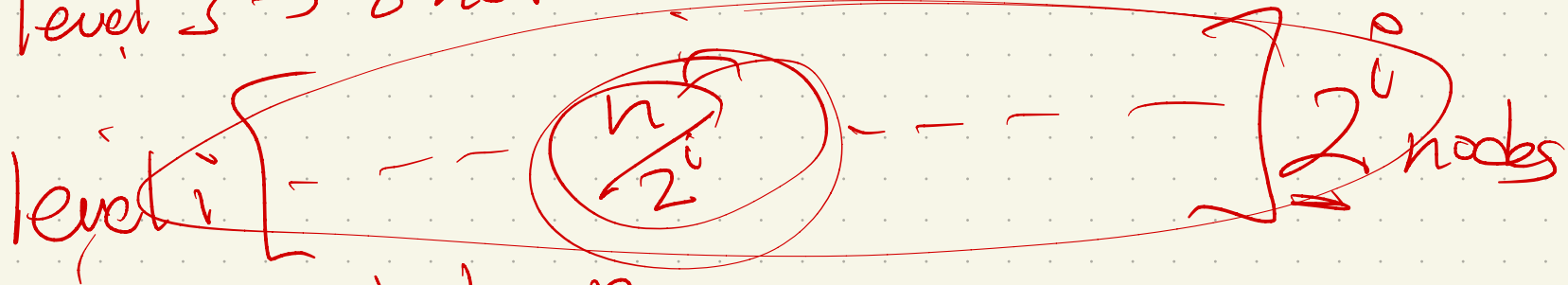




$$\rightarrow T(n) =$$



level 3 $\rightarrow$ 8 nodes



level $\log_2 n$

level $d = 1$

what is d ?

$$\frac{n}{2^d} \geq 1$$

$$\Rightarrow n \geq 2^d$$

$$\log_2 n \geq d$$

work
all nodes

$$\sum_{\text{level } i=0}^{\log_2 n}$$

$$T(n) = \sum_{\text{levels}} (\text{work inside node}) \times (\# \text{ of nodes})$$

$$= \sum_{i=0}^{\log_2 n} \left(\frac{n}{2^i} \right) \cdot 2^i$$

$$\stackrel{\text{math!}}{=} \sum_{i=0}^{\log_2 n} n = \underbrace{(n + n + \dots + n)}_{\log_2 n}$$

$$= n \log n$$

Idea:
rec

→ tree

"the stack",

←
Summation of all
work over alg's
lifetime

Recursion Trees:

Let's start with an example.

Suppose we have a function which:

- takes input of size n
- Makes 3 recursive calls to input of size $n/7$ each
- And has a double for loop inside

Then $T(n) = 3 \cdot T\left(\frac{n}{7}\right) + n^2$

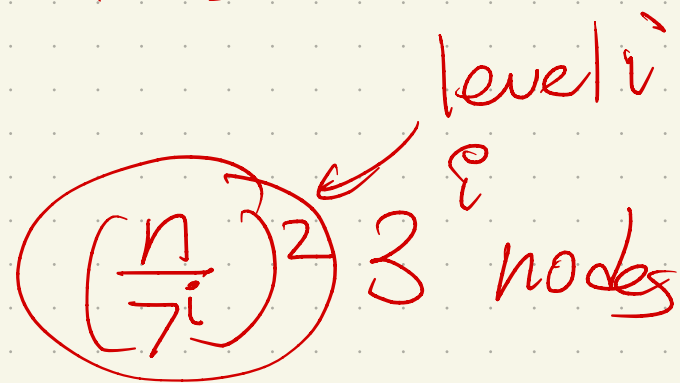
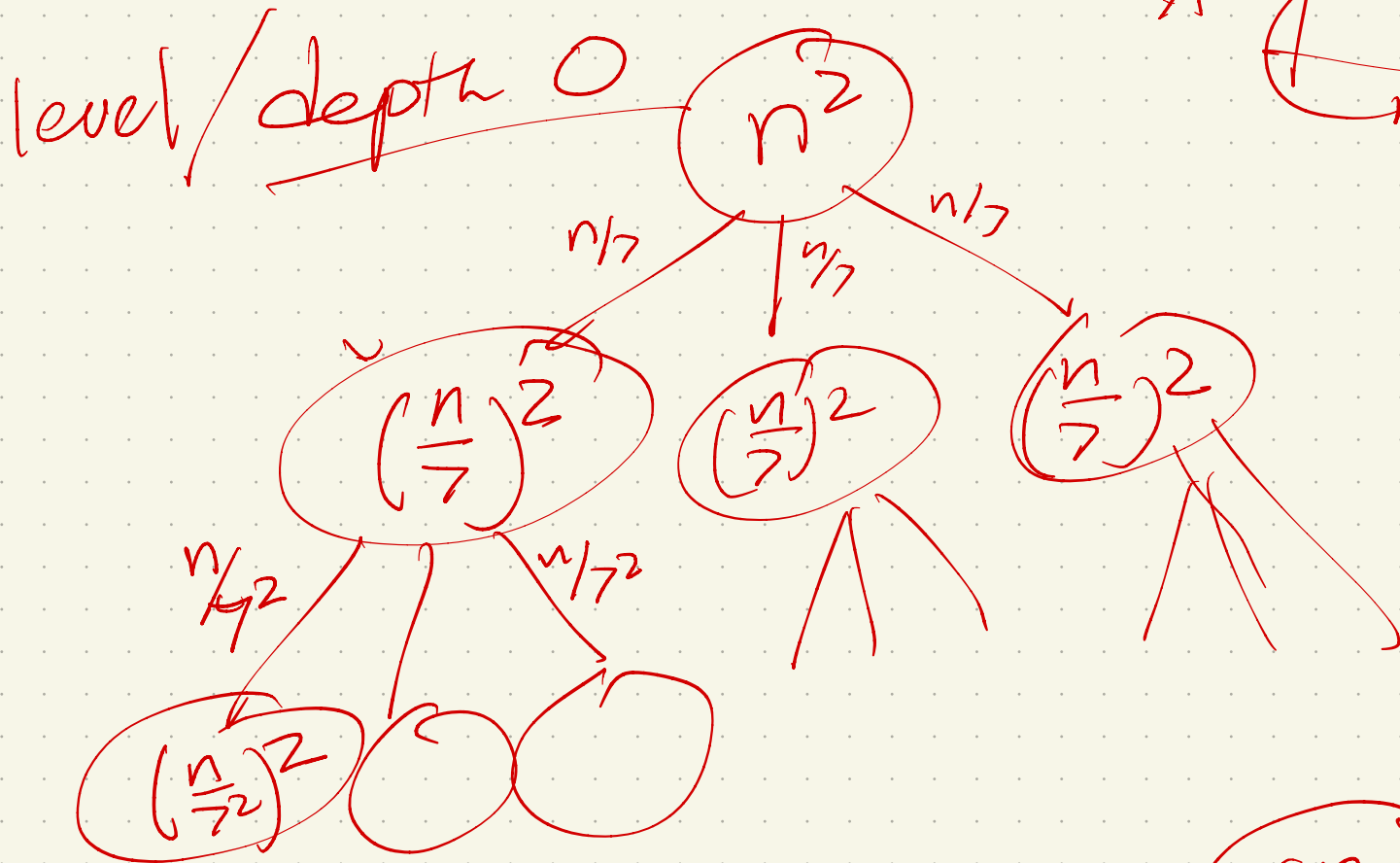
runtime
of alg

runtime?



Recursion tree

$$T(n) = 3T\left(\frac{n}{7}\right) + n^2$$



depth d

$\frac{n}{7^d} = 1 \Rightarrow n = 7^d \Rightarrow d = \log_7 n$

$$T(n) = \sum_{\text{levels } i} (\text{\# nodes in level } i) (\text{work per node})$$

$$d = \log_7 n$$

$$= \sum_{i=0}^{\log_7 n} (3^i) \cdot \left(\frac{n}{7^i}\right)^2 \quad \frac{1}{7^{2i}} = \frac{1}{(7^2)^i}$$

$$= \sum_{i=0}^{\log_7 n} 3^i \cdot n^2 \cdot \left(\frac{1}{7^{2i}}\right)$$

$$= n^2 \cdot \left[\sum_{i=0}^{\log_7 n} 3^i \cdot \frac{1}{49^i} \right]$$

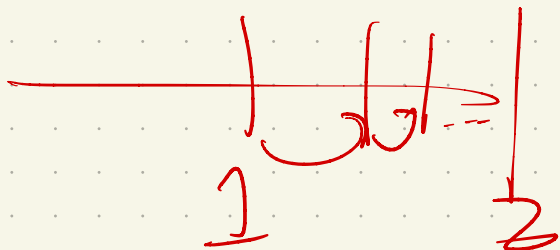
$$= n^2 \cdot \left[\sum_{i=0}^{\log_7 n} \left(\frac{3}{49}\right)^i \right] < n^2 \cdot \sum_{i=0}^{\infty} \left(\frac{3}{49}\right)^i$$

$$= n^2 \left(1 - \frac{1}{3/49}\right)$$

geometric series!

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \dots$$

$$\leq 2 = \frac{1}{1 - \frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2$$



$$\sum_{i=0}^{\infty} r^i < \frac{1}{1-r}$$

→ Here: $T(n) \leq n^2 \cdot \frac{49}{46} = O(n^2)$

$r < 1$

Next part: how to generalize?

$$T(n) = r T\left(\frac{n}{c}\right) + f(n)$$

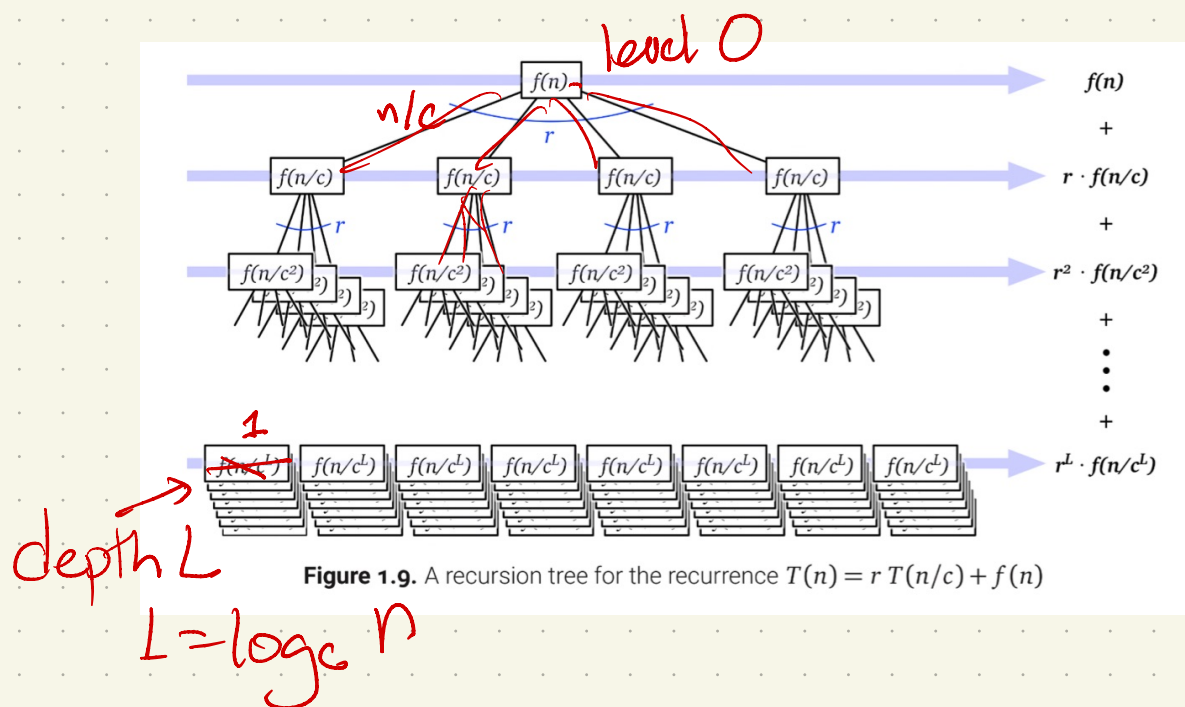
What it means:

Algorithm (n):
// code
for $i \leftarrow 1$ to r
 Algorithm ($\frac{n}{c}$)
// more code

total $f(n)$

Here, c & r are constants.

The tree:



Summing up:

$$\sum_{\text{levels}} (\text{work per node}) \times (\# \text{ nodes})$$

$$= \sum_{i=0}^{\log_c n} f\left(\frac{n}{c^i}\right) \cdot r^i$$

can we solve?

Geometric series:

$$\sum_{k=0}^n a \cdot \underbrace{b^k}_{r^k}$$

All depends on r :

if $b < 1$: 

$$\sum_{k=0}^n a \cdot \underbrace{b^k}_{r^k} < \sum_{k=0}^{\infty} a \cdot \underbrace{b^k}_{r^k} = \frac{a}{1-\underbrace{b}_{r}}$$

$$\underbrace{r}_{\left(\frac{3}{4}\right)}$$

$$\text{if } \underbrace{b}_{r} > 1: \quad = \frac{a(\underbrace{b^{n+1}}_{r^{n+1}} - 1)}{\underbrace{b}_{r} - 1}$$

Master Theorem:

Combining the three cases above gives us the following "master theorem".

Theorem 1 *The recurrence*

$$\begin{aligned} T(n) &= aT(n/b) + cn^k \\ T(1) &= c, \end{aligned}$$

where a , b , c , and k are all constants, solves to:

$$T(n) \in \Theta(n^k) \text{ if } a < b^k$$

$$T(n) \in \Theta(n^k \log n) \text{ if } a = b^k$$

$$\rightarrow T(n) \in \Theta(n^{\log_b a}) \text{ if } a > b^k$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n^{\frac{1}{4}}$$

2 vs 2^1 a^p b^p k
 ascending geom series
 descending
 $n > 1$
 $n < 1$
 $n = 1$

$$\begin{aligned} a &= 9 \\ b &= 3 \\ k &= 4 \end{aligned}$$

$$\begin{aligned} 9 &\text{ vs } 3^4 \\ 9 &< 3^4 \end{aligned}$$

$$\begin{aligned} k &= 1 \\ a &= 9 \\ b &= 3 \\ 9 &\text{ vs } 3^1 \\ &< \end{aligned}$$

THEOREM 2

MASTER THEOREM Let f be an increasing function that satisfies the recurrence relation

$$f(n) = af(n/b) + cn^d$$

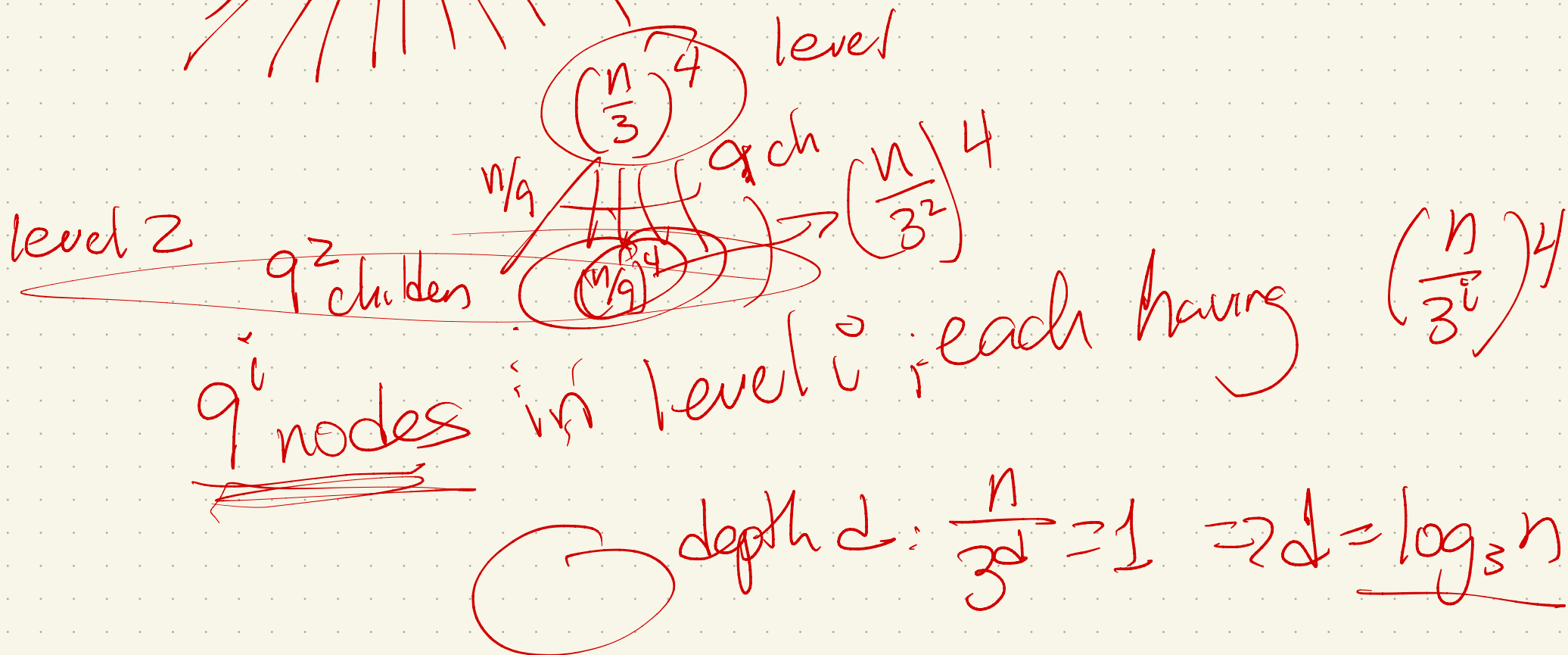
whenever $n = b^k$, where k is a positive integer, $a \geq 1$, b is an integer greater than 1, and c and d are real numbers with c positive and d nonnegative. Then

$$f(n) \text{ is } \begin{cases} O(n^d) & \text{if } a < b^d, \\ O(n^d \log n) & \text{if } a = b^d, \\ O(n^{\log_b a}) & \text{if } a > b^d. \end{cases}$$

Aside: when can't I use this?

More examples

$$T(n) = 9T\left(\frac{n}{3}\right) + n^4$$



$$\text{convert} \quad \sum_{\text{levels}} (\# \text{ nodes}) (\text{work in each})$$

$$= \sum_{i=0}^{\log_3 n} (9^i) \left(\frac{n}{3^i}\right)^4$$

$$= \sum_{i=0}^{\log_3 n} 9^i \cdot \frac{n^4}{3^{4i}} = n^4 \sum_{i=0}^{\log_3 n} \left(\frac{1}{3}\right)^i$$

$$= n^4 \left[\sum_{i=0}^{\log_3 n} \left(\frac{1}{3}\right)^i \right]$$

$$= n^4 \sum_{i=0}^{\log_3 n} \left(\frac{1}{3}\right)^i = n^4 \sum_{i=0}^{\infty} \left(\frac{1}{3}\right)^i$$

$$= n^4$$

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

$$T(n) = \sum_{i=0}^{\log_3 n} 9^i \cdot \left(\frac{n}{3^i}\right)$$

$$= n \cdot \left[\sum_{i=0}^{\log_3 n} \left(\frac{9}{3}\right)^i \right] = n \cdot \sum_{i=0}^{\log_3 n} 3^i$$

use geom series

$$\geq n \cdot \left[\frac{3^{\log_3 n + 1} - 1}{3 - 1} \right]$$

$$\Rightarrow n \cdot \left(\frac{3 \cdot 3^{\log_3 n} - 1}{2} \right) = n \cdot \left(\frac{3n - 1}{2} \right) = O(n^2)$$